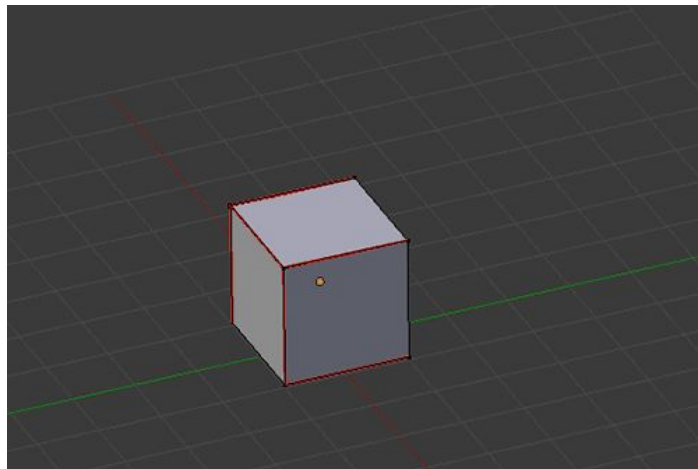
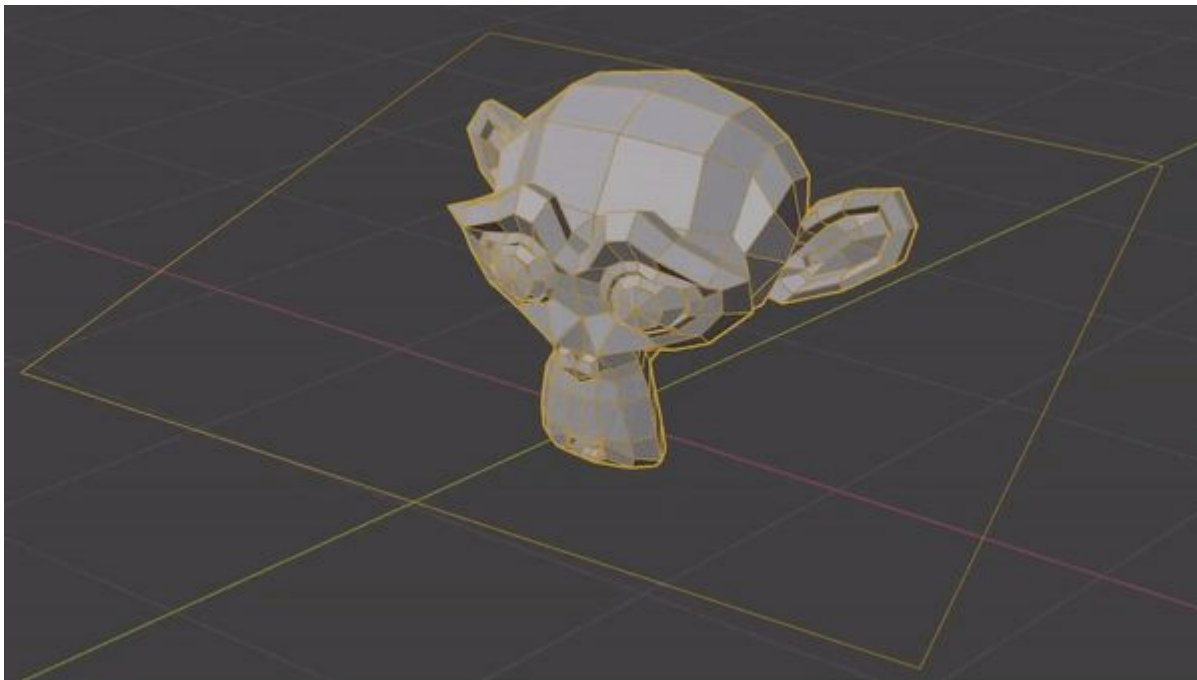


Introduction to Mesh Parameterization

Richard Liu



Introduction

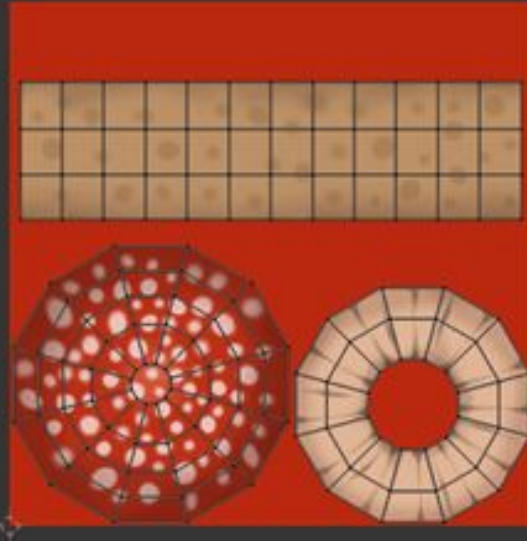


[Source](#)

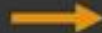
Motivating Application: Texture Mapping



3D modeling



UV Unwrapping



Texture painting

Roadmap

- Formal definition of parameterization and specification in mesh context

Roadmap

- Formal definition of parameterization and specification in mesh context
- Common approach to computing parameterizations with fixed boundary

Roadmap

- Formal definition of parameterization and specification in mesh context
- Common approach to computing parameterizations with fixed boundary
- Measure distortion of a parameterization

Roadmap

- Formal definition of parameterization and specification in mesh context
- Common approach to computing parameterizations with fixed boundary
- Measure distortion of a parameterization
- Least Squares Conformal Maps (LSCM)
 - **One of the options for the final project**

Source Material

Hormann, K., Polthier, K., Sheffer, A. “Mesh Parameterization: Theory and Practice”. Siggraph Asia 2008 Course.

<https://www.cs.princeton.edu/courses/archive/fall12/cos526/papers/hormann08.pdf>

Jacobson, A. Geometry processing course parameterization assignment.

<https://github.com/alecjacobson/geometry-processing-parameterization>

“Parameterization I”. Stanford CS-468.

https://graphics.stanford.edu/courses/cs468-10-fall/LectureSlides/12_Parameterization1.pdf

Sheffer, A., Praun, E., Rose, K. “Mesh Parameterization Methods and Their Applications”. Trends in CG and Vision 2006.

<https://people.eecs.berkeley.edu/~jrs/meshpapers/ShefferPraunRose.pdf>

Mathematical Foundations

A **mesh** is a discretization of a smooth surface into a set of vertices, edges, and faces.

Formally, a mesh is defined by the object $M = (\mathbf{V}, \mathbf{E}, \mathbf{F})$

Mathematical Foundations

A **mesh** is a discretization of a smooth surface into a set of vertices, edges, and faces.

Formally, a mesh is defined by the object $M = (\mathbf{V}, \mathbf{E}, \mathbf{F})$

Where

$$\mathbf{V} \subset \mathbb{R}^3$$

$$\mathbf{E} = \{(i, j) \mid \text{vertex } i \text{ is connected to vertex } j\}$$

$$\mathbf{F} = \{(i, j, k) \mid \text{vertices } i, j, k \text{ share a face}\}$$

Mathematical Foundations

A **mesh** is a discretization of a smooth surface into a set of vertices, edges, and faces.

Formally, a mesh is defined by the object $M = (\mathbf{V}, \mathbf{E}, \mathbf{F})$

Where

$$\mathbf{V} \subset \mathbb{R}^3$$

All our examples will involve triangle meshes, but the theory applies to general meshes.

$$\mathbf{E} = \{(i, j) \mid \text{vertex } i \text{ is connected to vertex } j\}$$

$$\mathbf{F} = \{(i, j, k) \mid \text{vertices } i, j, k \text{ share a face}\}$$

Mathematical Foundations

For a mesh $\mathbf{M}$, we define the parameterization as a function mapping vertices $\mathbf{V}$ to some 2D domain Ω

$$U : \mathbf{V} \subset \mathbb{R}^3 \rightarrow \Omega \subset \mathbb{R}^2$$

Mathematical Foundations

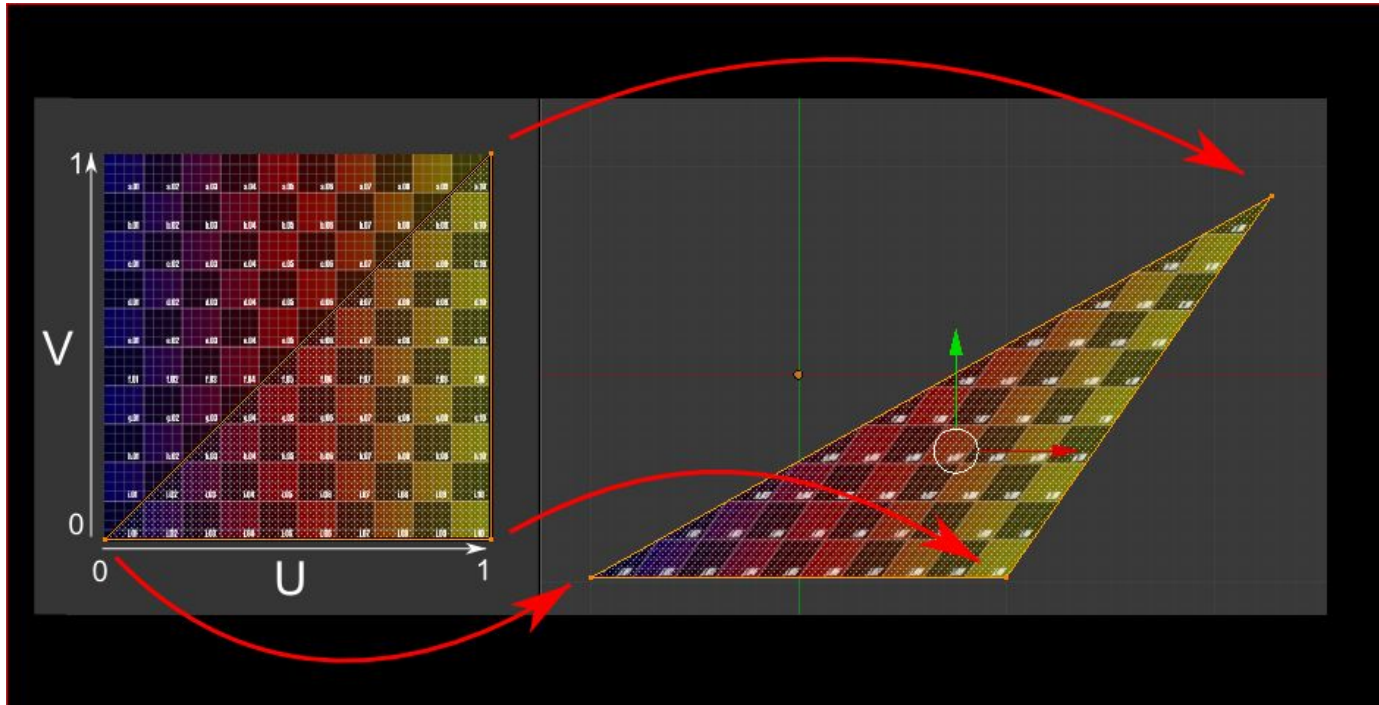
For a mesh $\mathbf{M}$, we define the parameterization as a function mapping vertices $\mathbf{V}$ to some 2D domain Ω

$$U : \mathbf{V} \subset \mathbb{R}^3 \rightarrow \Omega \subset \mathbb{R}^2$$

To put it simply, for every vertex v_i , we assign a 2D coordinate $U(v_i) = (u_i, v_i)$

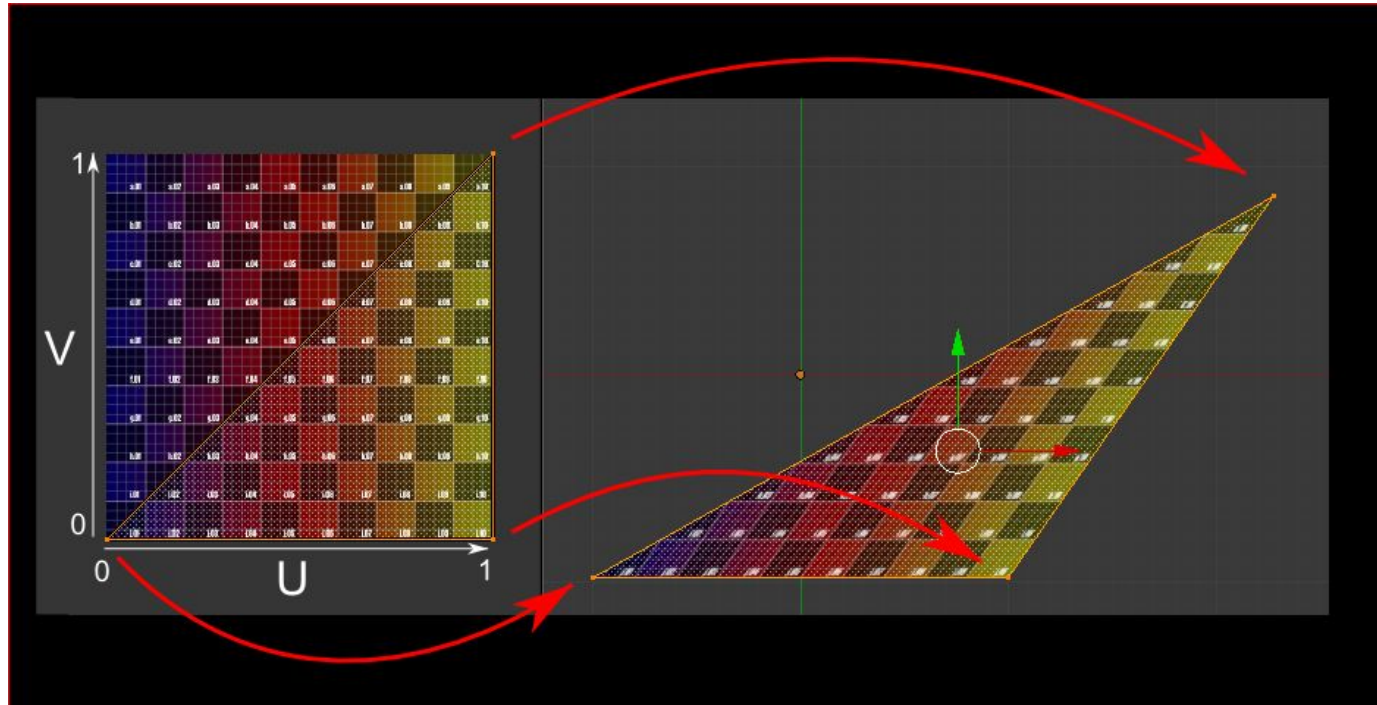
Mathematical Foundations

To put it simply, for every vertex v_i , we assign a 2D coordinate $U(v_i) = (u_i, v_i)$



Mathematical Foundations

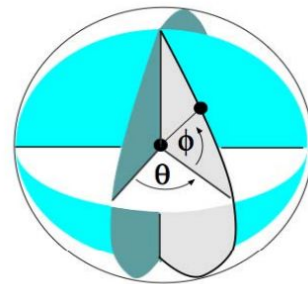
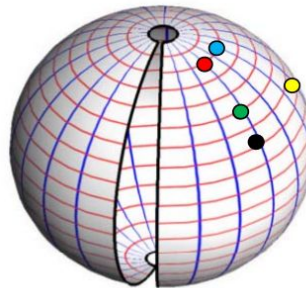
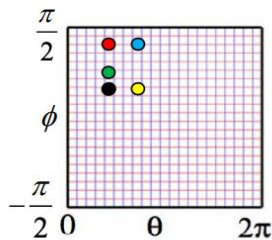
To put it simply, for every vertex v_i , we assign a 2D coordinate $U(v_i) = (u_i, v_i)$



The 2D axes are commonly referred to as “U” and “V” axes, hence the colloquial term **UV coordinates**

Common (Smooth) Parameterizations

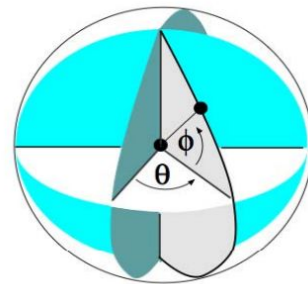
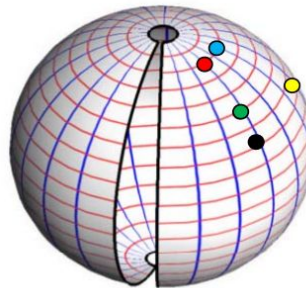
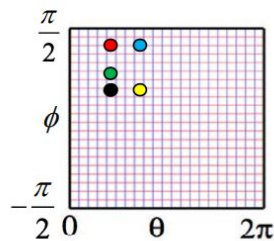
You've already seen
parameterizations before: spherical
coordinates!



Common (Smooth) Parameterizations

You've already seen
parameterizations before: spherical
coordinates!

For a sphere with radius R , we have
the parameterization function



$$U(x, y, z) = \left(\arctan \left(\frac{y}{x} \right), \arccos \left(\frac{z}{R} \right) \right) = (\theta, \phi)$$

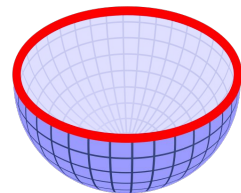
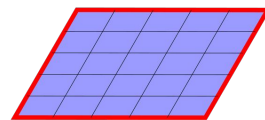
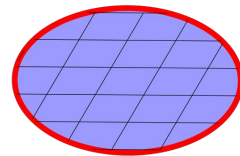
Mathematical Foundations

To compute the parameterization, we need to make an assumption of our surface.

Mathematical Foundations

To compute the parameterization, we need to make an assumption of our surface.

Definition. A **disk topology surface** is a surface with 1 boundary loop

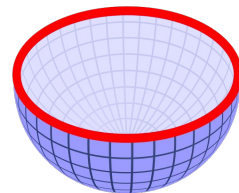
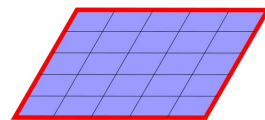
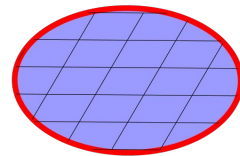


Mathematical Foundations

To compute the parameterization, we need to make an assumption of our surface.

Definition. A **disk topology surface** is a surface with 1 boundary loop

**Only disk topology surfaces admit a homeomorphism
(continuous and bijective map) to the (bounded) 2D plane.**



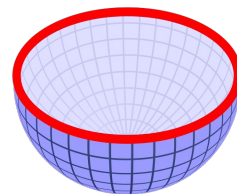
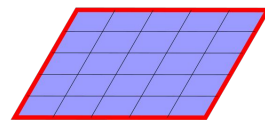
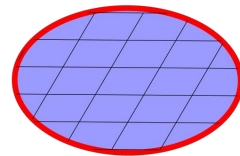
Mathematical Foundations

To compute the parameterization, we need to make an assumption of our surface.

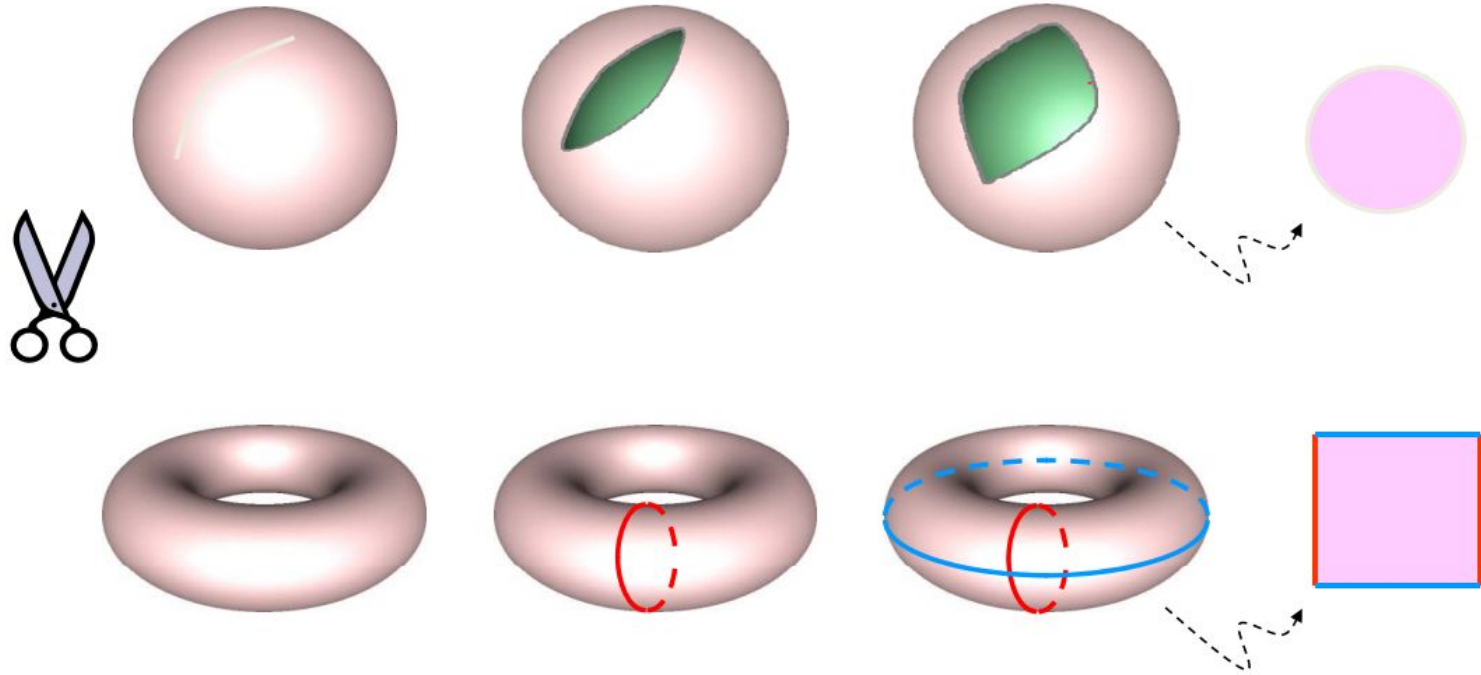
Definition. A **disk topology surface** is a surface with 1 boundary loop

Only disk topology surfaces admit a homeomorphism (continuous and bijective map) to the (bounded) 2D plane.

Given a closed (watertight) mesh, we can always introduce a cut to make it disk topology.



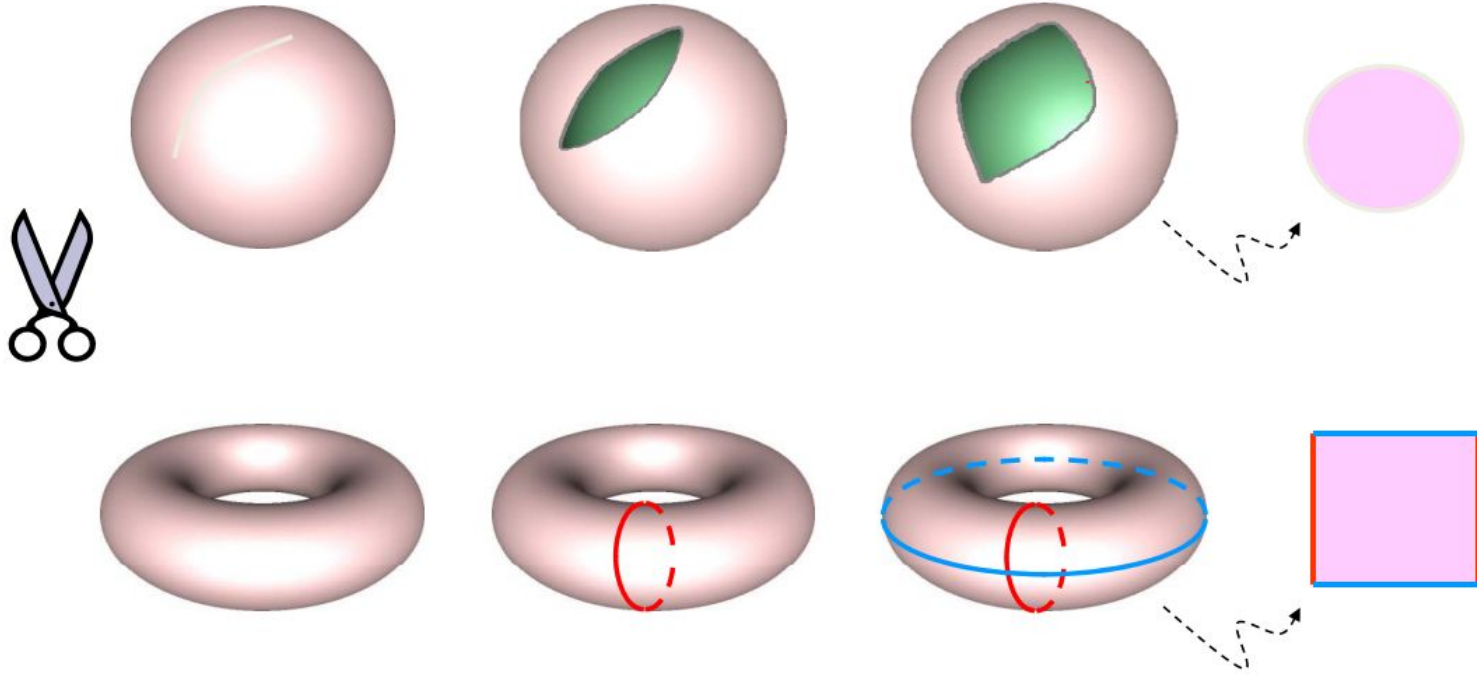
Disk Topology Detour



Creates artificial boundary

Disk Topology Detour

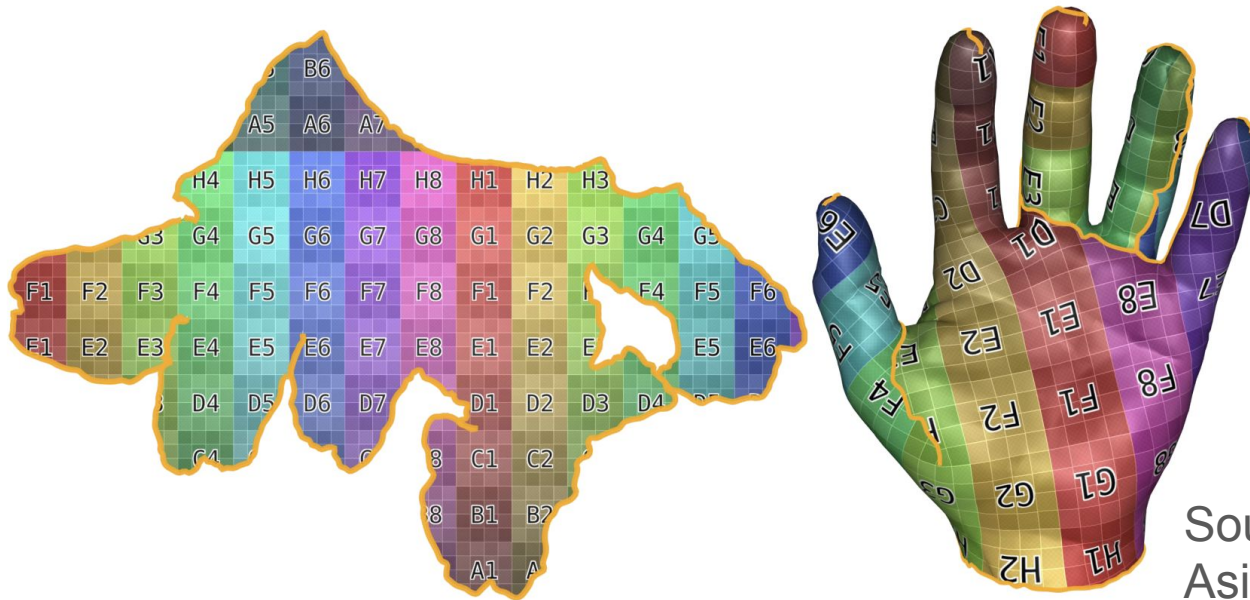
The problem with seams is they create **discontinuities** in the mapping.



Creates artificial boundary

Disk Topology Detour

The problem with seams is they create **discontinuities** in the mapping.



Source: Optcuts (Siggraph Asia 2018)

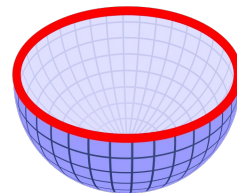
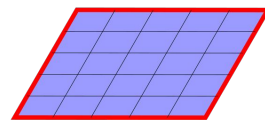
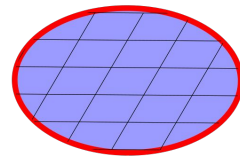
Mathematical Foundations

To compute the parameterization, we need to make an assumption of our surface.

Definition. A **disk topology surface** is a surface with 1 boundary loop

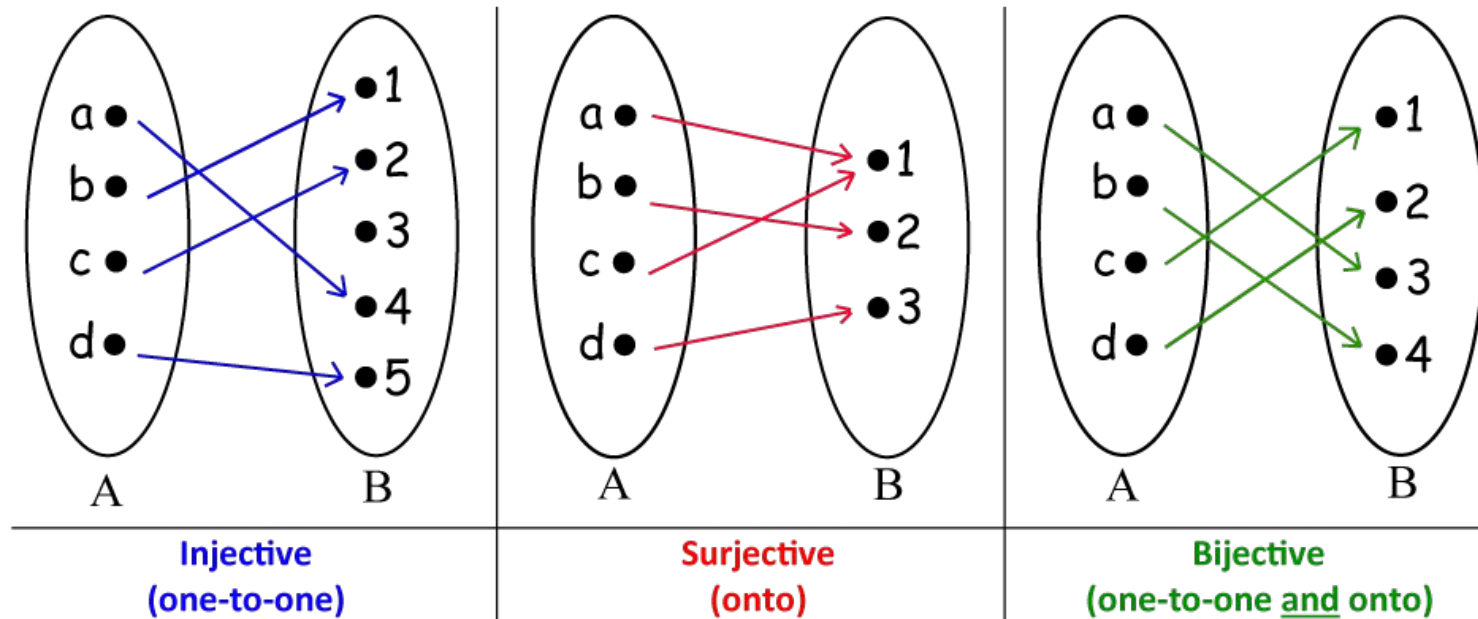
Only disk topology surfaces admit a homeomorphism (continuous and **bijjective map) to the (bounded) 2D plane.**

Given a closed (watertight) mesh, we can always introduce a cut to make it disk topology.



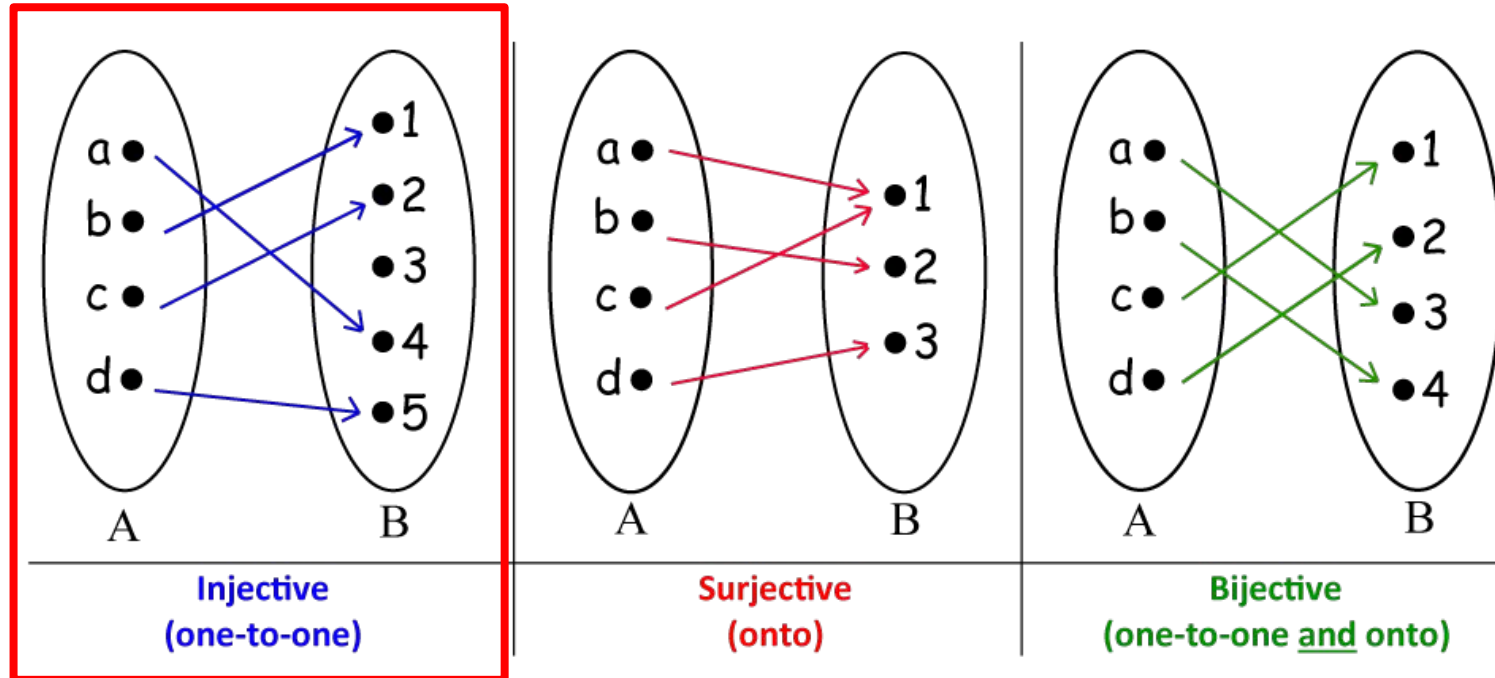
Mathematical Foundations

Bijection function definition



Mathematical Foundations

Bijection function definition



Injectivity in Texture Mapping



Image from Vallet and Levy, techreport INRIA

Injectivity in Texture Mapping



Image from Vallet and Levy, techreport INRIA

Mathematical Foundations

Traditionally, artists create UV parameterizations by hand – super time consuming and tedious.



Mathematical Foundations

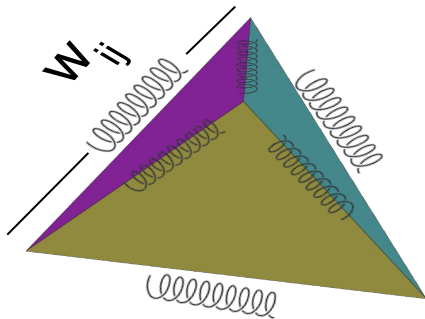
Traditionally, artists create UV parameterizations by hand – super time consuming and tedious.



Turns out there is a better way – compute the full parameterization by solving a system of linear equations.

Mathematical Foundations

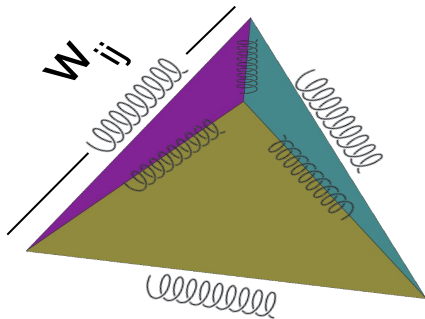
The better way: viewing the mesh surface as the graph of a **mass-spring system**.



Mathematical Foundations

The better way: viewing the mesh surface as the graph of a **mass-spring system**.

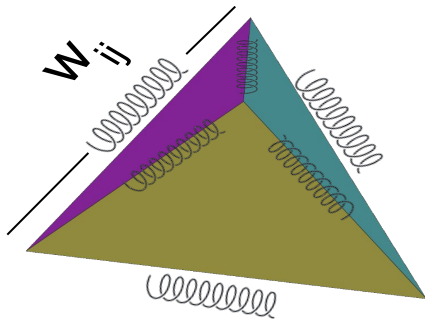
Mesh vertices are nodes (with equal mass), with each edge acting as a spring with some stiffness constant.



Mathematical Foundations

The better way: viewing the mesh surface as the graph of a **mass-spring system**.

Mesh vertices are nodes (with equal mass), with each edge acting as a spring with some stiffness constant.

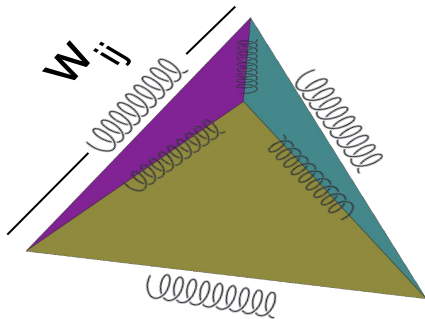


For disk topology meshes there are two key choices to make before computing a standard parameterization.

Mathematical Foundations

The better way: viewing the mesh surface as the graph of a **mass-spring system**.

Mesh vertices are nodes (with equal mass), with each edge acting as a spring with some stiffness constant.



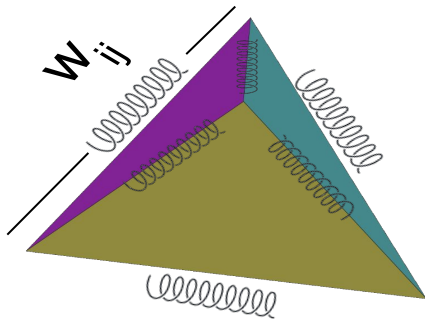
For disk topology meshes there are two key choices to make before computing a standard parameterization.

1. Fixing the positions of the boundary vertices in the 2D plane.

Mathematical Foundations

The better way: viewing the mesh surface as the graph of a **mass-spring system**.

Mesh vertices are nodes (with equal mass), with each edge acting as a spring with some stiffness constant.



For disk topology meshes there are two key choices to make before computing a standard parameterization.

1. Fixing the positions of the boundary vertices in the 2D plane.
2. Choosing the spring weights

Mathematical Foundations

Computing the parameterization becomes a minimization problem of the spring potential energies over 2D node positions.

$$\min_{\mathbf{U}} \sum_{\{i,j\} \in \mathbf{E}} w_{ij} ||\mathbf{u}_i - \mathbf{u}_j||^2.$$

We assume each edge is an ideal spring (zero rest length) with spring constant w_{ij} and all vertices are nodes with unit mass.

Mathematical Foundations

Computing the parameterization becomes a minimization problem of the spring potential energies over 2D node positions.

$$\min_{\mathbf{U}} \sum_{\{i,j\} \in \mathbf{E}} w_{ij} ||\mathbf{u}_i - \mathbf{u}_j||^2.$$

We assume each edge is an ideal spring (zero rest length) with spring constant w_{ij} and all vertices are nodes with unit mass.

One problem: there's a trivial solution! **set $\mathbf{U} = 0$**

Mathematical Foundations

Computing the parameterization becomes a minimization problem of the spring potential energies over 2D node positions.

$$\min_{\mathbf{U}} \sum_{\{i,j\} \in \mathbf{E}} w_{ij} ||\mathbf{u}_i - \mathbf{u}_j||^2.$$

We assume each edge is an ideal spring (zero rest length) with spring constant w_{ij} and all vertices are nodes with unit mass.

One problem: there's a trivial solution! **set $\mathbf{U} = 0$**

Solution: fix the positions of the boundary vertices

Mathematical Foundations

Two decisions we need to make before solving the minimization

$$\min_{\mathbf{U}} \sum_{\{i,j\} \in \mathbf{E}} w_{ij} ||\mathbf{u}_i - \mathbf{u}_j||^2.$$

Mathematical Foundations

Two decisions we need to make before solving the minimization

1. How to choose the shape of the boundary? (Assign $\mathbf{U}(v_i)$ for $v_i \in \partial\mathbf{V}$)

$$\min_{\mathbf{U}} \sum_{\{i,j\} \in \mathbf{E}} w_{ij} \|\mathbf{u}_i - \mathbf{u}_j\|^2.$$

Mathematical Foundations

Two decisions we need to make before solving the minimization

1. How to choose the shape of the boundary? (Assign $\mathbf{U}(v_i)$ for $v_i \in \partial\mathbf{V}$)
2. How to choose the spring coefficients w_{ij} ? (Laplacian weights)

$$\min_{\mathbf{U}} \sum_{\{i,j\} \in \mathbf{E}} w_{ij} \|\mathbf{u}_i - \mathbf{u}_j\|^2.$$

Mathematical Foundations

Two decisions we need to make before solving the minimization

1. How to choose the shape of the boundary? (Assign $\mathbf{U}(v_i)$ for $v_i \in \partial\mathbf{V}$)
2. How to choose the spring coefficients w_{ij} ? (Laplacian weights)

[Tutte 1963/Floater 1997] If pinned boundary is **convex** and **weights are positive**, then the solution is guaranteed to be injective.

$$\min_{\mathbf{U}} \sum_{\{i,j\} \in \mathbf{E}} w_{ij} ||\mathbf{u}_i - \mathbf{u}_j||^2.$$

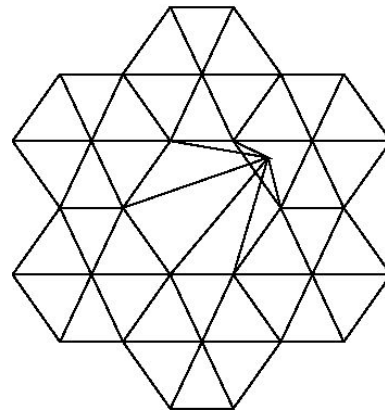
Mathematical Foundations

Two decisions we need to make before solving the minimization

1. How to choose the shape of the boundary? (Assign $\mathbf{U}(v_i)$ for $v_i \in \partial\mathbf{V}$)
2. How to choose the spring coefficients w_{ij} ? (Laplacian weights)

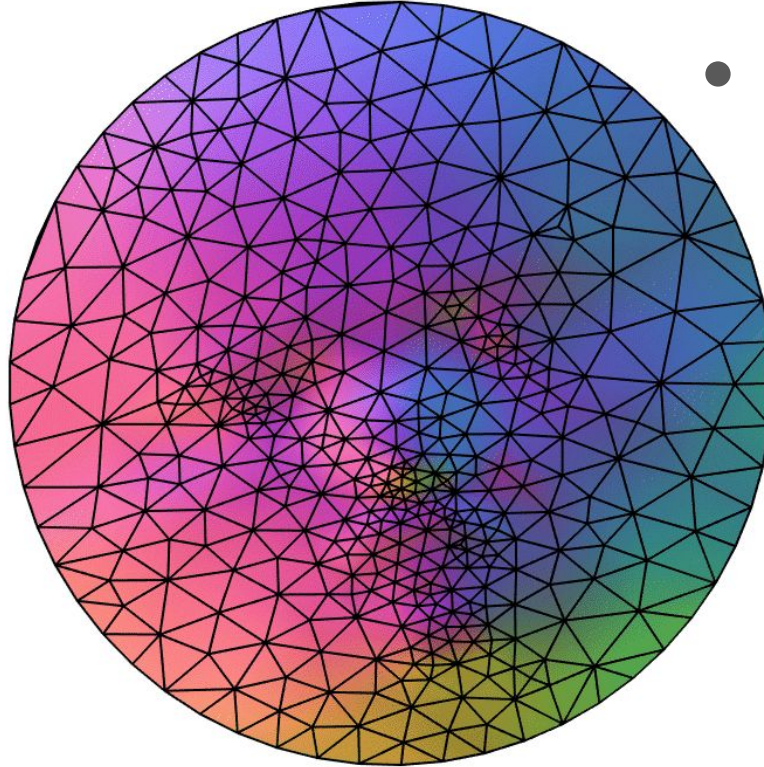
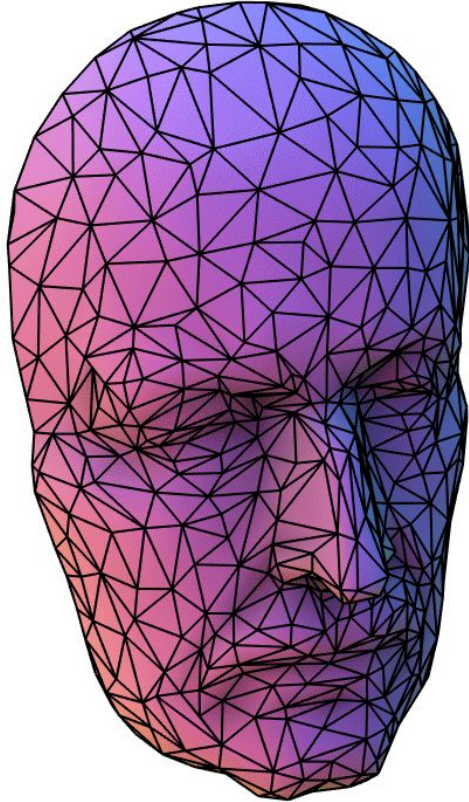
[Tutte 1963/Floater 1997] If pinned boundary is **convex** and **weights are positive**, then the solution is guaranteed to be injective.

$$\min_{\mathbf{U}} \sum_{\{i,j\} \in \mathbf{E}} w_{ij} ||\mathbf{u}_i - \mathbf{u}_j||^2.$$



Triangle Flip

Mathematical Foundations



Tutte embedding

- Boundary is mapped to circle
- All weights are 1

Source:

<https://github.com/alecjacobson/geometry-processing-parameterization?tab=readme-ov-file>

Mathematical Foundations

How to solve?

$$E = \min_{\mathbf{U}} \sum_{\{i,j\} \in \mathbf{E}} w_{ij} ||\mathbf{u}_i - \mathbf{u}_j||^2$$

Mathematical Foundations

How to solve?

$$E = \min_{\mathbf{U}} \sum_{\{i,j\} \in \mathbf{E}} w_{ij} ||\mathbf{u}_i - \mathbf{u}_j||^2$$

$$\frac{\partial E}{\partial \mathbf{u}_i} = \sum_{j \in N_i} 2 * w_{ij} (\mathbf{u}_i - \mathbf{u}_j) = 0$$

Where N_i indexes all of the neighbors of vertex i .

Mathematical Foundations

How to solve?

$$E = \min_{\mathbf{U}} \sum_{\{i,j\} \in \mathbf{E}} w_{ij} ||\mathbf{u}_i - \mathbf{u}_j||^2$$

$$\frac{\partial E}{\partial \mathbf{u}_i} = \sum_{j \in N_i} 2 * w_{ij} (\mathbf{u}_i - \mathbf{u}_j) = 0$$

Where N_i indexes all of the neighbors of vertex i .

Fix boundary vertices and solve each equation for remaining $\mathbf{u}_i$

Mathematical Foundations

We call the general class of methods which follow this procedure **fixed boundary methods**.

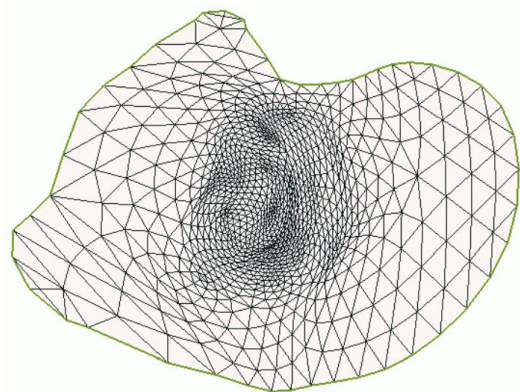
$$\min_{\mathbf{U}} \sum_{\{i,j\} \in \mathbf{E}} w_{ij} ||\mathbf{u}_i - \mathbf{u}_j||^2.$$

Mathematical Foundations

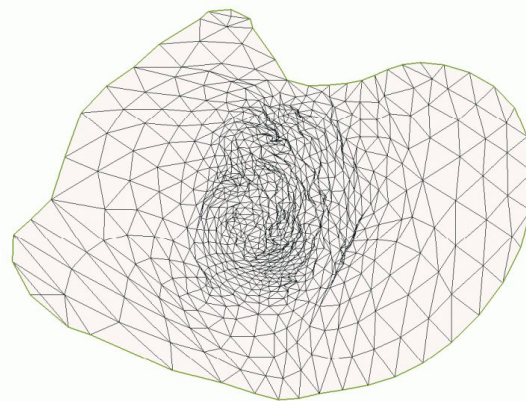
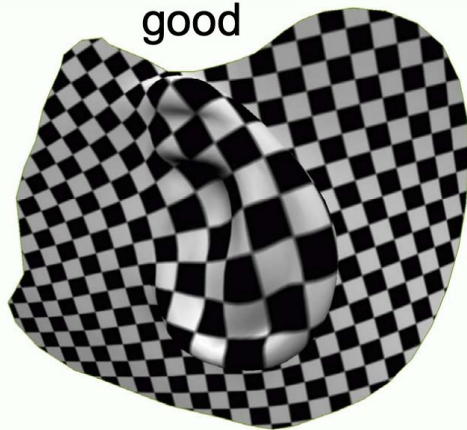
Desirable properties of parameterizations

- Injectivity
- Cut (discontinuity) minimization
- **Low distortion**

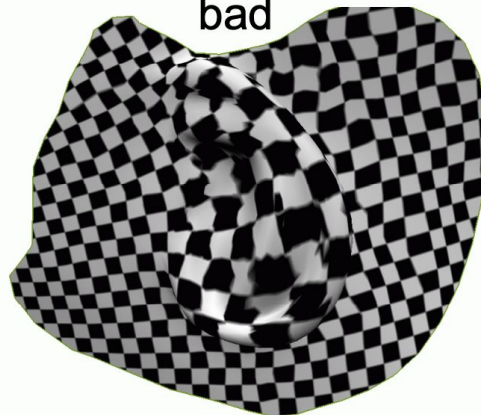
Distortion



good



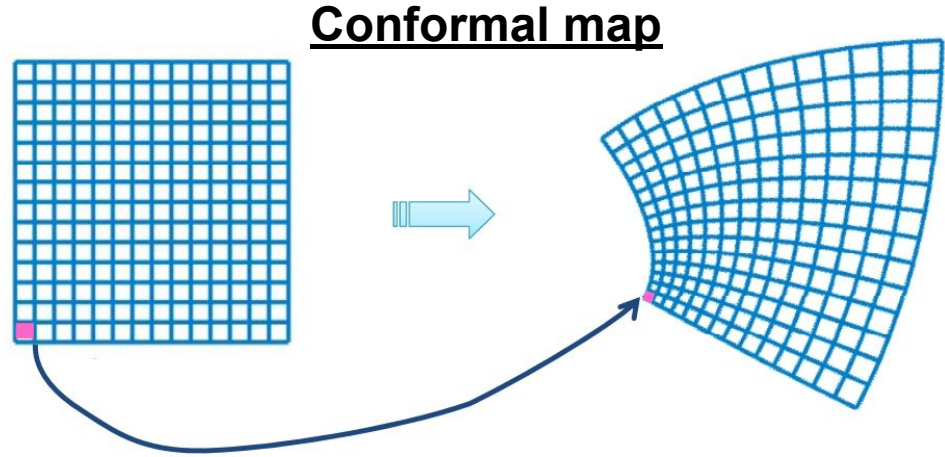
bad



Distortion

Two types of distortion

- **Conformal**: angle distortion



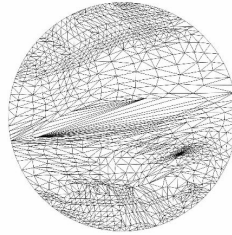
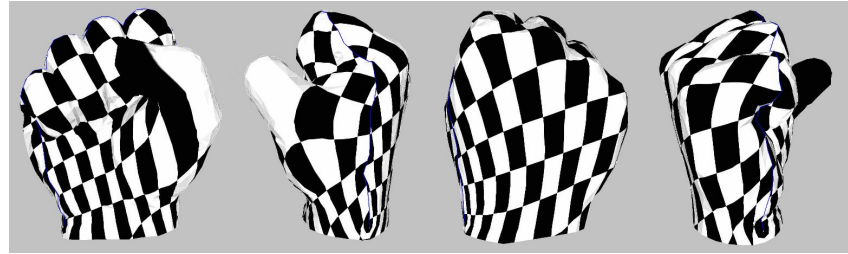
Similarity = Rotation + Scale
Preserves angles

Distortion

Two types of distortion

- Conformal: angle distortion
- **Authalic**: area distortion

Authalic map



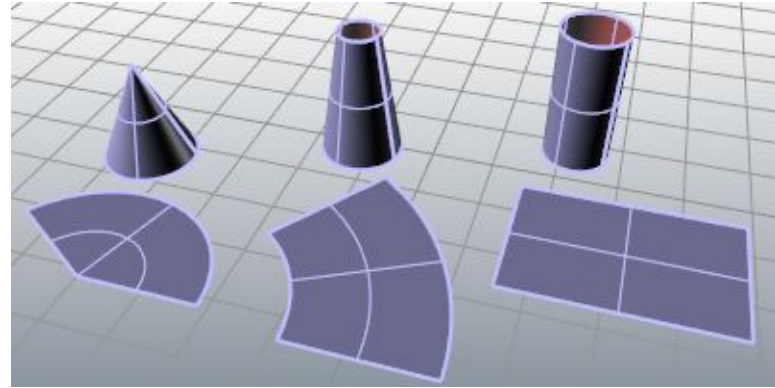
Preserves area

Distortion

Two types of distortion

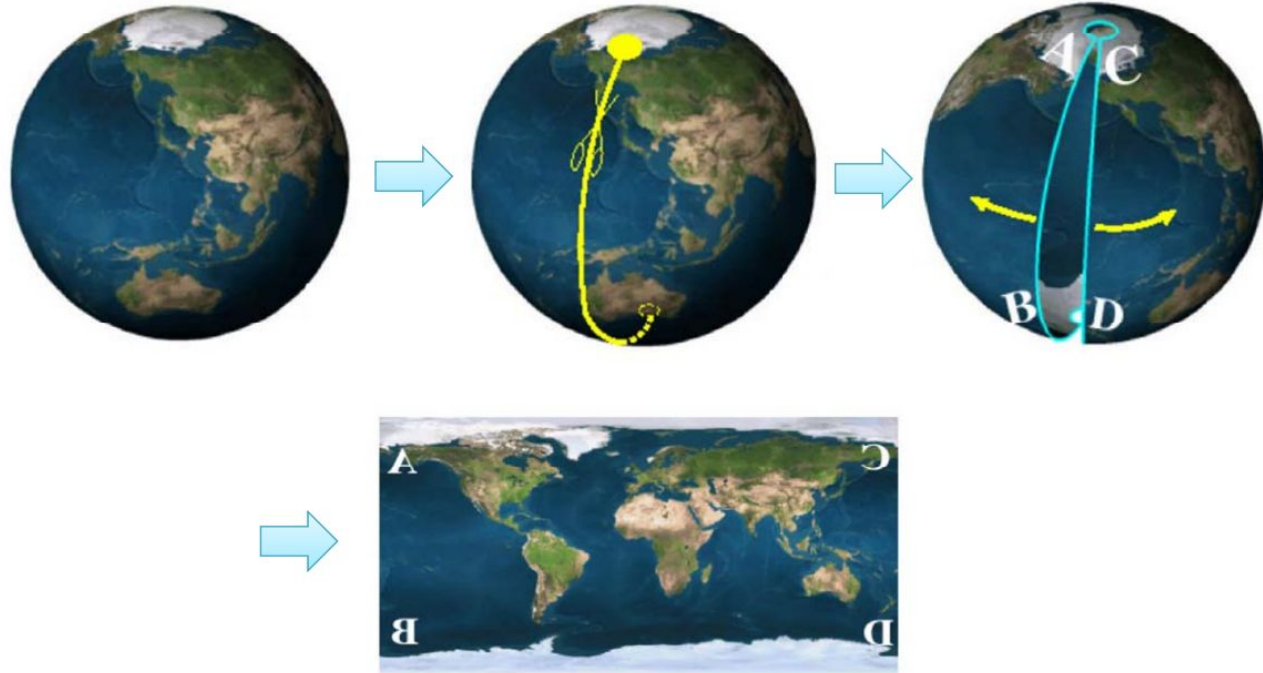
- Conformal: angle distortion
- Authalic: area distortion

Maps which preserve both **area (authalic)** and **angles (conformal)** are **isometric** (length-preserving)



Distortion

Most classic examples of distortion come from cartography



Distortion

Most classic examples of distortion come from cartography



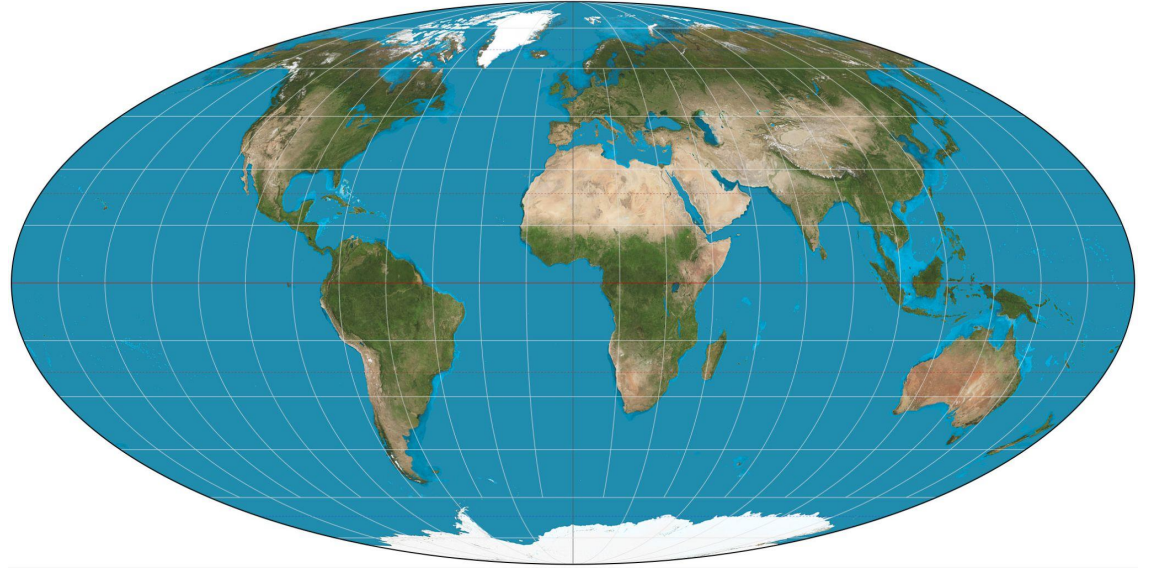
Mercator projection
(conformal)



Source: Jakub Nowosad
<https://commons.wikimedia.org/w/index.php?curid=73955926>

Distortion

Most classic examples of distortion come from cartography



Mollweide projection
(equiareal/authalic)

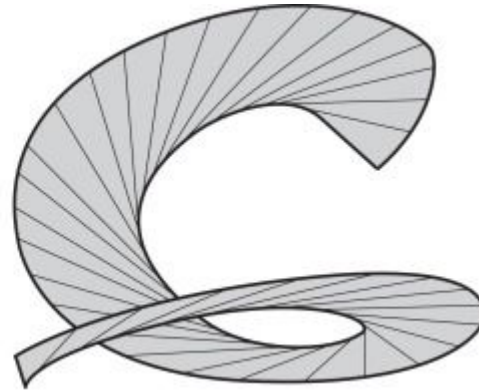
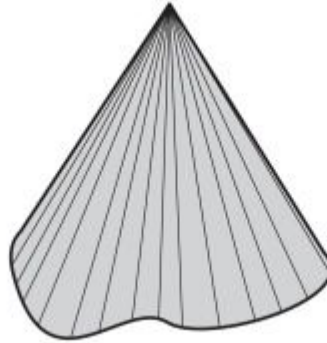
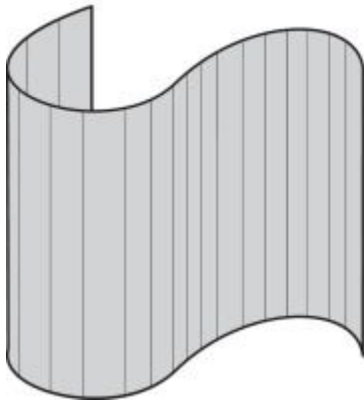
Distortion

Why are there no isometric (distortionless) maps of the world?

Distortion

Why are there no isometric (distortionless) maps of the world?

Theorema Egregium [Gauss 1827] Only surfaces with zero Gaussian curvature (developable) can be isometrically flattened to the plane.



Distortion

Ok if most parameterizations create distortion how to measure it?

Distortion

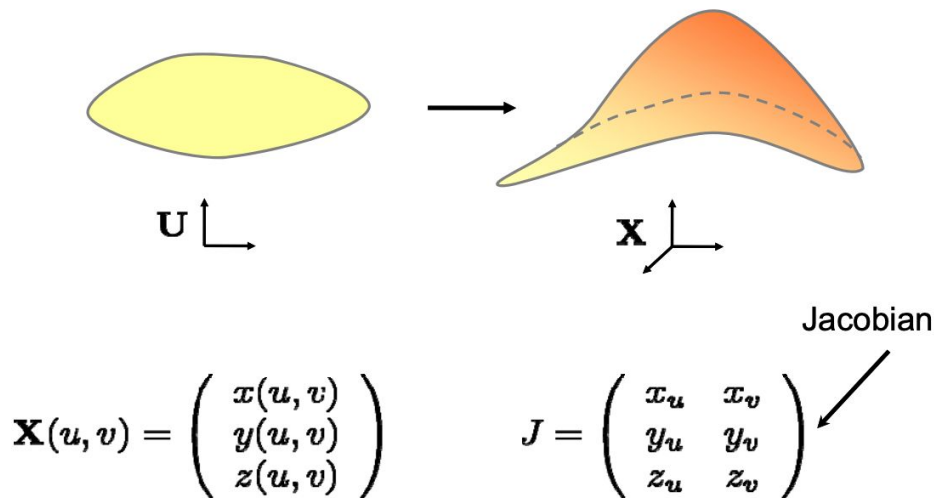
Ok if most parameterizations create distortion how to measure it?

The Jacobian tells you everything you need to know about distortion.

Distortion

Ok if most parameterizations create distortion how to measure it?

The Jacobian tells you everything you need to know about distortion.



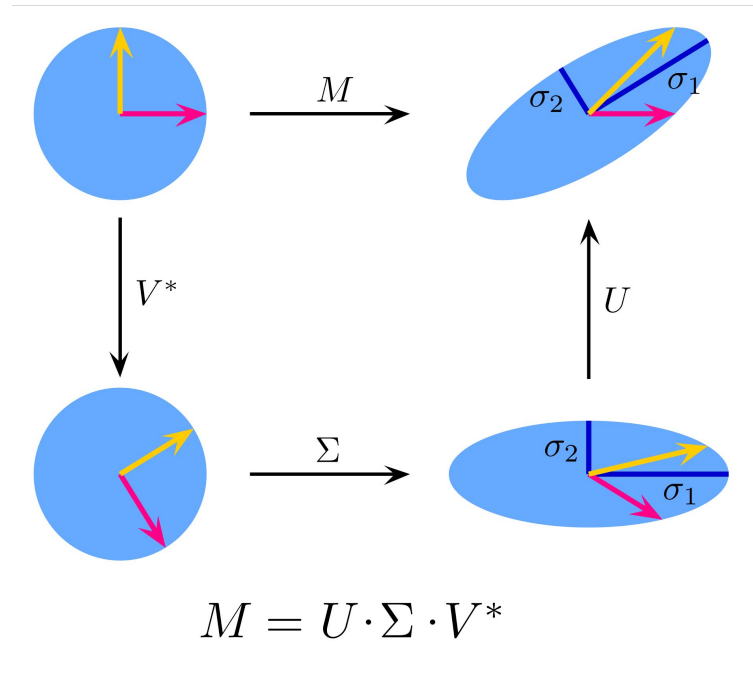
Distortion

Specifically, the singular values of the Jacobian tell you everything

$$J_f = U\Sigma V^T = U \begin{pmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{pmatrix} V^T$$

Distortion

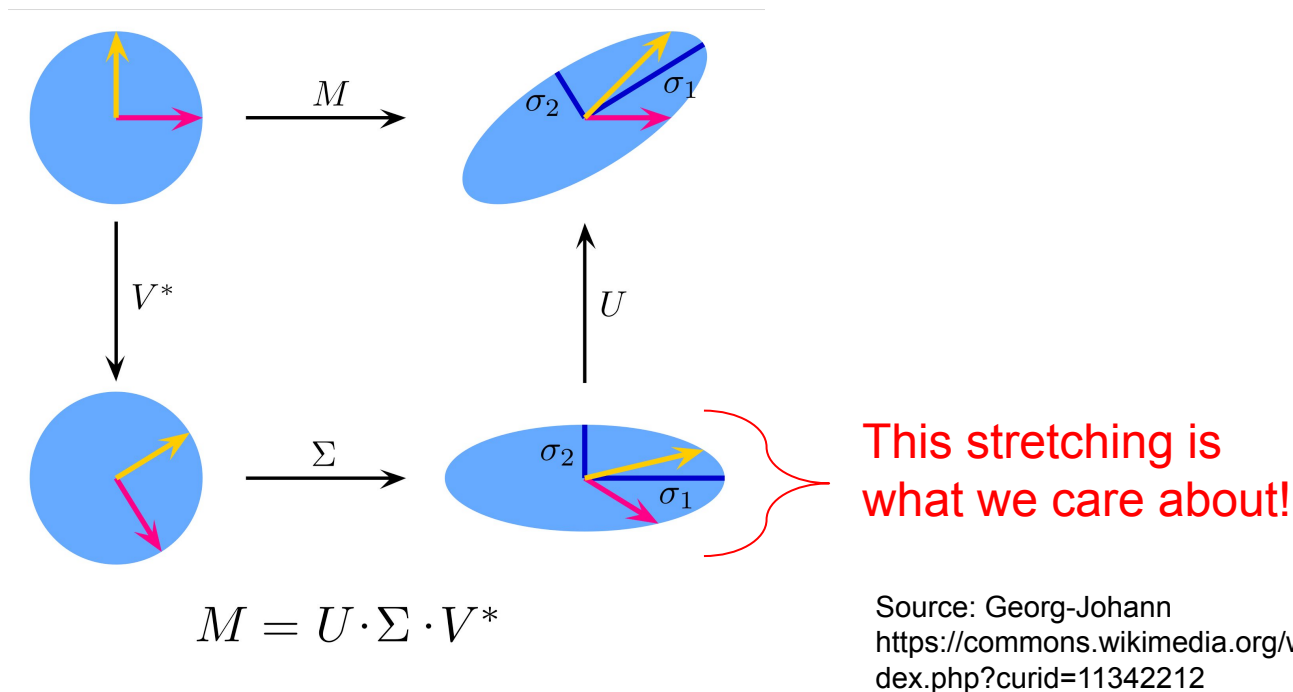
Super fast singular value decomposition (SVD) primer



Source: Georg-Johann
<https://commons.wikimedia.org/w/index.php?curid=11342212>

Distortion

Super fast singular value decomposition (SVD) primer



Source: Georg-Johann
<https://commons.wikimedia.org/w/index.php?curid=11342212>

Distortion

Specifically, the singular values of the Jacobian tell you everything

$$J_f = U\Sigma V^T = U \begin{pmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{pmatrix} V^T$$

Distortion

Specifically, the singular values of the Jacobian tell you everything

$$J_f = U\Sigma V^T = U \begin{pmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{pmatrix} V^T$$

$$f \text{ is equiareal or area-preserving} \iff \sigma_1 \sigma_2 = 1$$

Distortion

Specifically, the singular values of the Jacobian tell you everything

$$J_f = U\Sigma V^T = U \begin{pmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{pmatrix} V^T$$

$$f \text{ is equiareal or area-preserving} \iff \sigma_1 \sigma_2 = 1$$

$$f \text{ is conformal or angle-preserving} \iff \sigma_1 = \sigma_2$$

Distortion

Specifically, the singular values of the Jacobian tell you everything

$$J_f = U\Sigma V^T = U \begin{pmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{pmatrix} V^T$$

$$f \text{ is equiareal or area-preserving} \iff \sigma_1 \sigma_2 = 1$$

$$f \text{ is conformal or angle-preserving} \iff \sigma_1 = \sigma_2$$

$$f \text{ is isometric or length-preserving} \iff \sigma_1 = \sigma_2 = 1$$

Distortion

Practically, we can find the Jacobians of the UV map assignment $\mathbf{U}$ using the mesh gradient operator $\mathbf{G} \in \mathbb{R}^{3|F| \times V}$

Distortion

Practically, we can find the Jacobians of the UV map assignment $\mathbf{U}$ using the mesh gradient operator $\mathbf{G} \in \mathbb{R}^{3|F| \times V}$

$$\mathbf{GU} = \mathbf{J} \in \mathbb{R}^{3|F| \times 2}$$

Distortion

Practically, we can find the Jacobians of the UV map assignment $\mathbf{U}$ using the mesh gradient operator $\mathbf{G} \in \mathbb{R}^{3|F| \times V}$

$$\mathbf{G}\mathbf{U} = \mathbf{J} \in \mathbb{R}^{3|F| \times 2}$$

How $\mathbf{G}$ is derived is beyond lecture scope (see [the libigl documentation](#) if interested).

Least Squares Conformal Maps (LSCM)

Least Squares Conformal Maps for Automatic Texture Atlas Generation

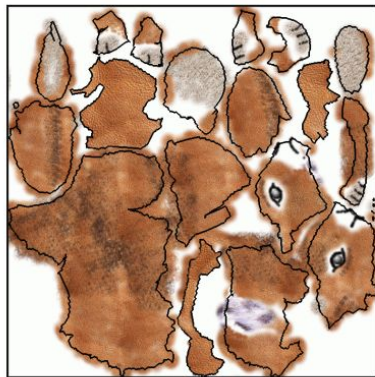
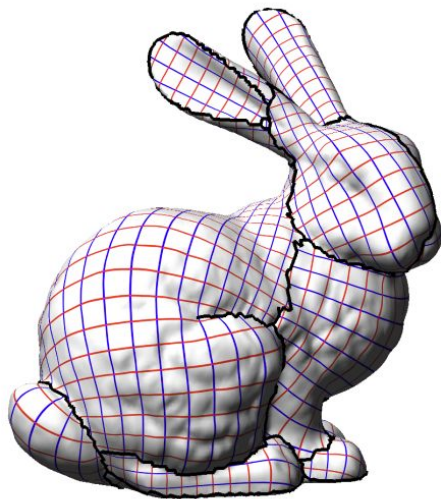
Bruno Lévy

Sylvain Petitjean

Nicolas Ray

Jérôme Maillot*

ISA (Inria Lorraine and CNRS), France



LSCM. (Levy et al. 2002) The least squares conformal maps method seeks to minimize the following conformal energy

$$E_{LSCM} = E_C = \frac{1}{2} \int_S ||f_v - \text{rot}_{90}(f_u X)||^2 dp = \frac{(\sigma_1 - \sigma_2)^2}{2}$$

LSCM. (Levy et al. 2002) The least squares conformal maps method seeks to minimize the following conformal energy

$$E_{LSCM} = E_C = \frac{1}{2} \int_S ||f_v - \text{rot}_{90}(f_u X)||^2 dp = \frac{(\sigma_1 - \sigma_2)^2}{2}$$

Intuition: We want the singular values to be equal ($\sigma_1 = \sigma_2$), aka **conformality**

Paper's perspective: Cauchy-Riemann equations from complex analysis.

Theorem

If a function $f(z) = u(x, y) + iv(x, y)$ is complex differentiable (holomorphic), then it satisfies the partial differential equations

$$\begin{aligned}\frac{\partial u}{\partial x} &= \frac{\partial v}{\partial y} \\ \frac{\partial u}{\partial y} &= -\frac{\partial v}{\partial x}\end{aligned}$$

Holomorphic implies conformality. This is equivalent to a condition on the Jacobian.

Forget the fancy math! This is equivalent to a simple condition on the Jacobian.

$$i \frac{\partial f}{\partial x} = \frac{\partial f}{\partial y} \Leftrightarrow \begin{vmatrix} a & -b \\ b & a \end{vmatrix}$$

Forget the fancy math! This is equivalent to a simple condition on the Jacobian.

$$i \frac{\partial f}{\partial x} = \frac{\partial f}{\partial y} \Leftrightarrow \begin{vmatrix} a & -b \\ b & a \end{vmatrix}$$

LSCM - Least Squares Conformal Map

We want the
Jacobian

$$\begin{pmatrix} \partial_x u & \partial_y u \\ \partial_x v & \partial_y v \end{pmatrix}$$

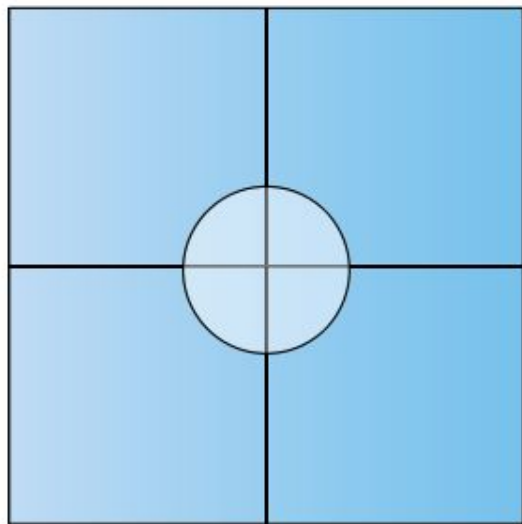
to be a
similarity matrix

$$\begin{pmatrix} \alpha & -\beta \\ \beta & \alpha \end{pmatrix}$$

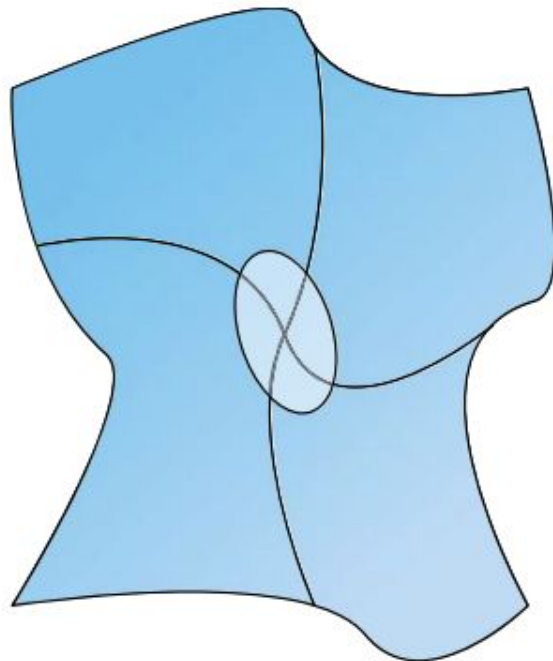
$$\mathcal{D}_{\text{LSCM}} = (\partial_x u - \partial_y v)^2 + (\partial_y u + \partial_x v)^2$$

Jacobian looks a little funny ...

$$\begin{pmatrix} u_x & u_y \\ v_x & v_y \end{pmatrix} = \begin{pmatrix} \nabla u \\ \nabla v \end{pmatrix}$$



$$\mathbf{f}(x, y) = (u, v)$$

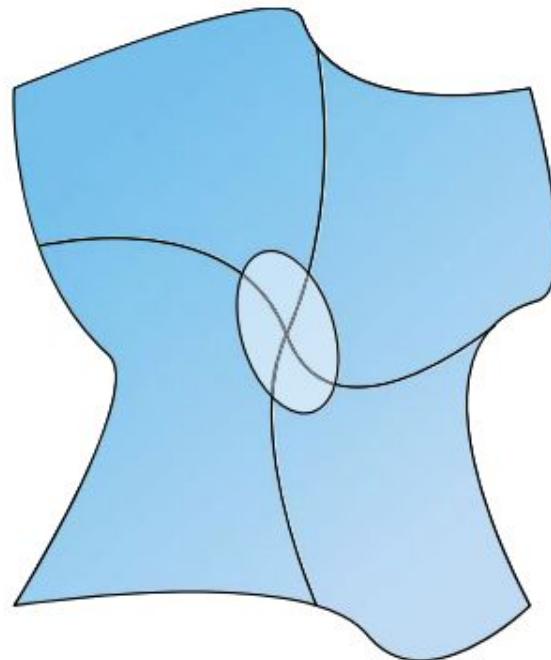
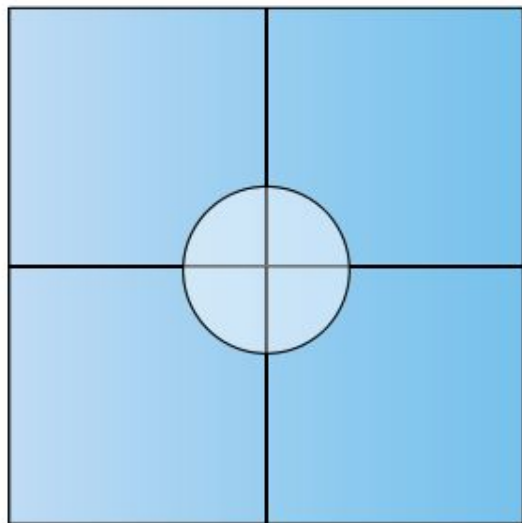
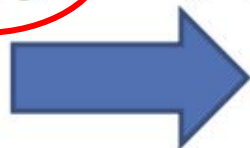


Jacobian looks a little funny ...

$$\begin{pmatrix} u_x & u_y \\ v_x & v_y \end{pmatrix} = \begin{pmatrix} \nabla u \\ \nabla v \end{pmatrix}$$

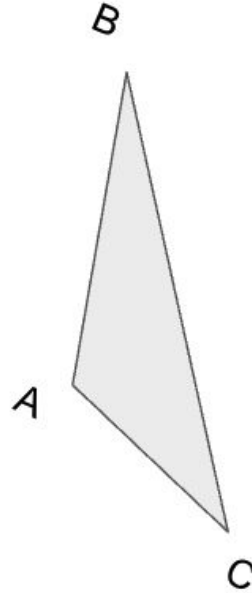
Where's z???

$$\mathbf{f}(x, y) = (u, v)$$



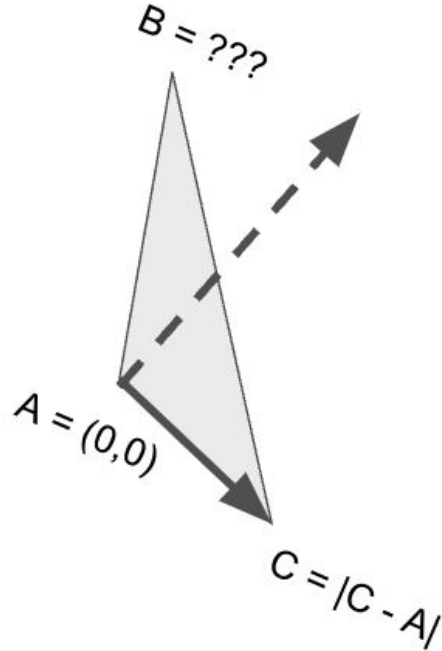
Local Basis Projection

Every triangle in the mesh can be defined by its own (local) coordinate system.



Local Basis Projection

Every triangle in the mesh can be defined by its own (local) coordinate system.



Find the local coordinates of B yourself.

Pinning

LSCM - Least Squares Conformal Map

We want the
Jacobian

$$\begin{pmatrix} \partial_x u & \partial_y u \\ \partial_x v & \partial_y v \end{pmatrix}$$

to be a
similarity matrix

$$\begin{pmatrix} \alpha & -\beta \\ \beta & \alpha \end{pmatrix}$$

$$\mathcal{D}_{\text{LSCM}} = (\partial_x u - \partial_y v)^2 + (\partial_y u + \partial_x v)^2$$

Pinning

LSCM - Least Squares Conformal Map

We want the
Jacobian

$$\begin{pmatrix} \partial_x u & \partial_y u \\ \partial_x v & \partial_y v \end{pmatrix}$$

to be a
similarity matrix

$$\begin{pmatrix} \alpha & -\beta \\ \beta & \alpha \end{pmatrix}$$

$$\mathcal{D}_{\text{LSCM}} = (\partial_x u - \partial_y v)^2 + (\partial_y u + \partial_x v)^2$$

There's a degenerate solution here. What trivial function has all partial derivatives 0?

Pinning

LSCM - Least Squares Conformal Map

We want the
Jacobian

$$\begin{pmatrix} \partial_x u & \partial_y u \\ \partial_x v & \partial_y v \end{pmatrix}$$

to be a
similarity matrix

$$\begin{pmatrix} \alpha & -\beta \\ \beta & \alpha \end{pmatrix}$$

$$\mathcal{D}_{\text{LSCM}} = (\partial_x u - \partial_y v)^2 + (\partial_y u + \partial_x v)^2$$

There's a degenerate solution here. What trivial function has all partial derivatives 0?

Constant function

Pinning

LSCM - Least Squares Conformal Map

We want the
Jacobian

$$\begin{pmatrix} \partial_x u & \partial_y u \\ \partial_x v & \partial_y v \end{pmatrix}$$

to be a
similarity matrix

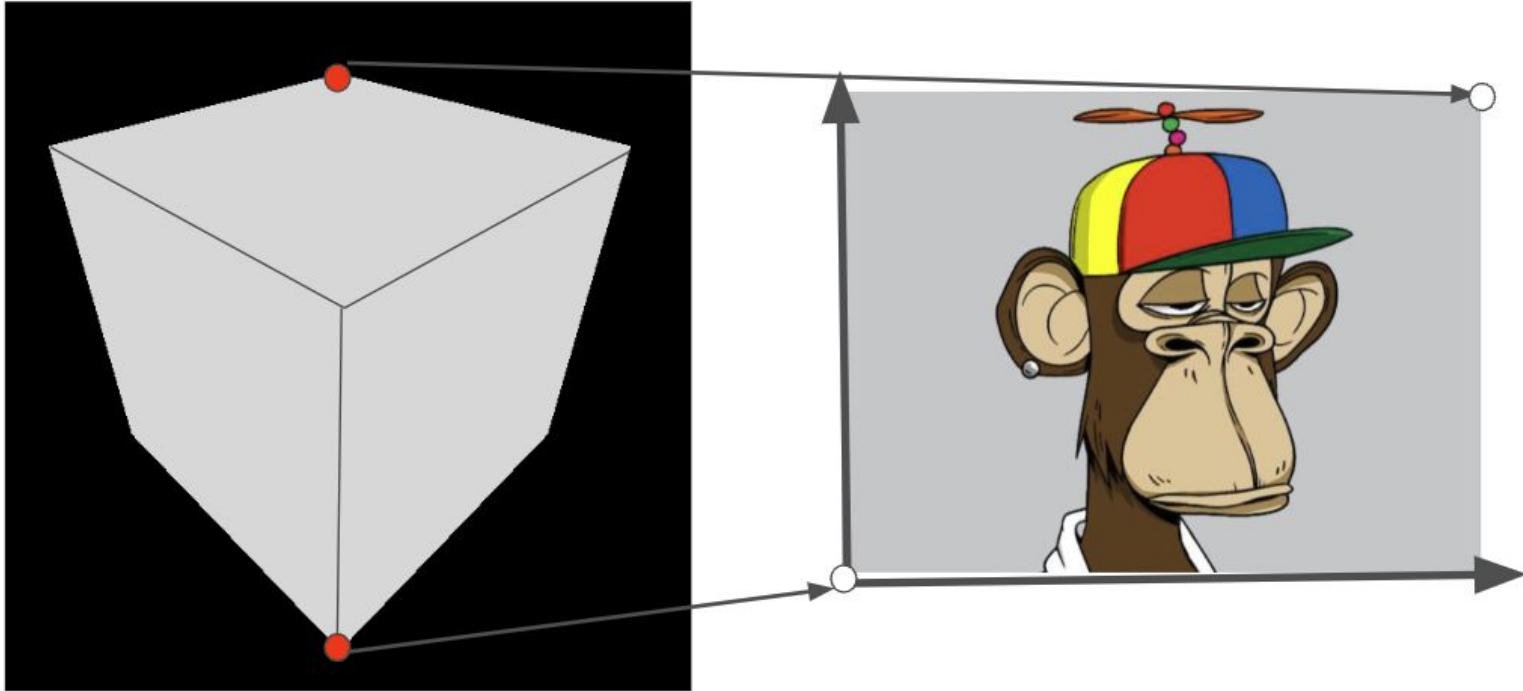
$$\begin{pmatrix} \alpha & -\beta \\ \beta & \alpha \end{pmatrix}$$

$$\mathcal{D}_{\text{LSCM}} = (\partial_x u - \partial_y v)^2 + (\partial_y u + \partial_x v)^2$$

For the system to have a unique solution, we need to set boundary constraints. Specifically, we need to pin the UV coordinates of two vertices.

Pinning

Most common way to pin is to send two vertices to the corners of the texel grid.



LSCM

Let's now set up the system in matrix form.

Rewriting the objective function with only real matrices and vectors yields

$$C(\mathbf{x}) = \|\mathcal{A}\mathbf{x} - \mathbf{b}\|^2, \quad (4)$$

with

$$\mathcal{A} = \begin{pmatrix} \mathcal{M}_f^1 & -\mathcal{M}_f^2 \\ \mathcal{M}_f^2 & \mathcal{M}_f^1 \end{pmatrix}, \quad \mathbf{b} = - \begin{pmatrix} \mathcal{M}_p^1 & -\mathcal{M}_p^2 \\ \mathcal{M}_p^2 & \mathcal{M}_p^1 \end{pmatrix} \begin{pmatrix} \mathbf{U}_p^1 \\ \mathbf{U}_p^2 \end{pmatrix},$$

LSCM

Let's now set up the system in matrix form.

Rewriting the objective function with only real matrices and vectors yields

$$C(\mathbf{x}) = \|\mathcal{A}\mathbf{x} - \mathbf{b}\|^2, \quad (4)$$

with

$$\mathcal{A} = \begin{pmatrix} \mathcal{M}_f^1 & -\mathcal{M}_f^2 \\ \mathcal{M}_f^2 & \mathcal{M}_f^1 \end{pmatrix}, \quad \mathbf{b} = - \begin{pmatrix} \mathcal{M}_p^1 & -\mathcal{M}_p^2 \\ \mathcal{M}_p^2 & \mathcal{M}_p^1 \end{pmatrix} \begin{pmatrix} \mathbf{U}_p^1 \\ \mathbf{U}_p^2 \end{pmatrix},$$

**Look
familiar?**

We want the
Jacobian

$$\begin{pmatrix} \partial_x u & \partial_y u \\ \partial_x v & \partial_y v \end{pmatrix}$$

to be a
similarity matrix

$$\begin{pmatrix} \alpha & -\beta \\ \beta & \alpha \end{pmatrix}$$

LSCM

$$\mathcal{M}^1 = \left(\begin{array}{c} \text{Assuming face } f_i \text{ with vertex positions } (x_1, y_1), (x_2, y_2), (x_3, y_3). \\ M_{ij}^1 = \begin{cases} x_3 - x_2 & j = \text{vertex1} \\ x_1 - x_3 & j = \text{vertex2} \\ x_2 - x_1 & j = \text{vertex3} \end{cases} \end{array} \right)$$

faces

vertices

LSCM

$$\mathcal{M}^2 =$$

vertices

faces

Assuming face f_i with vertex positions $(x_1, y_1), (x_2, y_2), (x_3, y_3)$.

$$M_{ij}^2 = \begin{cases} y_3 - y_2 & j = \text{vertex1} \\ y_1 - y_3 & j = \text{vertex2} \\ y_2 - y_1 & j = \text{vertex3} \end{cases}$$

LSCM

$$\mathcal{M}_f^1 \equiv \mathcal{M}^1 \left(\begin{array}{c} \circ \\ \circ \end{array} \right) \text{Indices of non-pinned vertices}$$

$$\mathcal{M}_p^1 \equiv \mathcal{M}^1 \left(\text{Indices of pinned vertices} \right)$$

$$\mathcal{M}_f^2 \equiv \mathcal{M}^2 \left(\begin{array}{c} \circ \\ \circ \end{array} \right) \text{Indices of non-pinned vertices}$$

$$\mathcal{M}_p^2 \equiv \mathcal{M}^2 \left(\begin{array}{c} \circ \end{array} \right) \text{Indices of pinned vertices}$$

$$\begin{pmatrix} \mathbf{U}_p^1 \\ \mathbf{U}_p^2 \end{pmatrix} \equiv \begin{array}{|c|} \hline 0 \\ 1 \\ 0 \\ 1 \\ \hline \end{array}$$

LSCM Pseudocode

def LSCM(V: vertices, F: faces):

- ① Choose two vertices (b1, b2) to pin
 - Use vertices that are far apart for better results!

LSCM Pseudocode

def LSCM(V: vertices, F: faces):

- ① Choose two vertices (b1, b2) to pin
 - Use vertices that are far apart for better results!
- ② For each triangle, convert the incident vertices to local coordinates

LSCM Pseudocode

def LSCM(V: vertices, F: faces):

- ① Choose two vertices (b1, b2) to pin
 - Use vertices that are far apart for better results!
- ② For each triangle, convert the incident vertices to local coordinates
- ③ Construct the **A** and **b** matrices from (4).

LSCM Pseudocode

def LSCM(V: vertices, F: faces):

- ① Choose two vertices (b1, b2) to pin
 - Use vertices that are far apart for better results!
- ② For each triangle, convert the incident vertices to local coordinates
- ③ Construct the **A** and **b** matrices from (4).
- ④ Solve the least squares equation for **x** in (4).
 - This will give you back your UV coordinates in a 1D vector in the form $(u_1, u_2, \dots, u_{|V|}, v_1, v_2, \dots, v_{|V|})$ **with the pinned vertices removed**
 - Make sure to **add the pinned vertices back** and reshape!

Thank you!

